Phase 1.0

1. **Requirements**

Functional Requirements

Registration & Validation

· The system should allow students to self-register using their student ID, full name, date of birth, university email address and password.

· The system should allow communication with the University's Student Academic Records Management System (SARMS) API endpoint to validate student information.

· The system should allow validation of student information against the SARMS before completing registration.

· The system should allow registration only if the student exists in SARMS and the submitted details match official records.

· The system should allow rejection of any student whose details do not exist or do not match SARMS records.

· The system should allow sending of an account verification link to the student's university email.

· The system should allow students to verify their accounts by clicking an account confirmation link sent to their university email

Authentication & Security

· The system should allow registered users to log in using their university email and password.

· The system should allow secure password storage using hashing techniques.

· The system should allow the use of two-factor authentication (2FA) during login

· The system should allow generation and validation of a time-limited One-Time Password (OTP) for 2FA.

· The system should allow users to request a password reset link sent to their university email.

· The system should allow users to securely reset their passwords using a time-limited reset token.

Membership Payment & Activation

· The system should allow students to pay an annual membership fee of P100 through a secure online payment gateway.

· The system should allow automatic activation of membership upon successful payment confirmation.

· The system should allow the system to record membership start date and expiry date (12 months from payment).

· The system should allow blocking of membership activation if payment fails.

· The system should allow members to view their payment history.

· The system should allow generation of a payment receipt upon successful transaction.

Membership Lifecycle Management

· The system should allow membership to remain valid for 12 months from the date of payment.

· The system should allow automatic sending of renewal reminders two months before membership expiry.

· The system should allow automatic blocking of accounts once membership expires.

· The system should allow blocked users to initiate renewal by submitting their university email address.

· The system should allow generation of a renewal link sent to the user's university email.

· The system should allow the renewal process to be exposed through an API endpoint.

· The system should allow members to view their membership status

Equipment Booking

- The system should allow active members to book club equipment online.
- The system should allow members to select equipment, booking date, start time, and end time.
- The system should allow validation to prevent overlapping bookings for the same equipment.
- The system should allow rejection of booking requests that conflict with existing bookings.
- The system should allow sending of booking confirmation emails.
- The system should allow sending of booking reminder emails two hours before the booking begins.
- The system should allow members to cancel upcoming bookings.
- The system should allow members to view their booking history.

Notifications

- The system should allow sending of all system notifications via email, including account verification, password reset, 2FA OTP, payment confirmation, booking reminders, and renewal reminders.
- The system should allow future integration with SMS and WhatsApp notification services.

Membership Card

- The system should allow generation of a digital membership card for each active member.
- The system should allow the membership card to display member name, student ID, membership ID, and validity period.
- The system should allow administrators to print or download membership cards in PDF format.

Administration

- The system should allow administrators to view and manage all registered members.

· The system should allow administrators to manage equipment records.

· The system should allow administrators to view and manage booking records.

· The system should allow administrators to manually block or unblock member accounts.

· The system should allow administrators to generate management reports.

· The system should allow logging of user activities and administrative actions for auditing purposes.

- The system should allow administrators to add new equipment.
- The system should allow administrators to update equipment details.
- The system should allow administrators to deactivate unavailable equipment.

Reporting

· The system should allow generation of membership reports, including total members, active members, expired members, and new registrations.

· The system should allow generation of financial reports, including total revenue collected and payment status summaries.

· The system should allow generation of equipment usage reports and booking statistics.

· The system should allow export of reports in PDF and Excel formats.

Non-Functional Requirements

Security

· The system should ensure secure communication using HTTPS encryption.

· The system should ensure secure password storage using strong hashing algorithms.

· The system should ensure role-based access control for students and administrators.

· The system should ensure protection against SQL injection, cross-site scripting (XSS), and cross-site request forgery (CSRF).

· The system should ensure secure integration with the SARMS API and payment gateway.

Performance

- The system should respond to user requests within 2 seconds under normal load.
- The system should handle at least 1,000 concurrent users.
- The system should ensure booking validation occurs in less than 1 second.

Availability & Reliability

- The system should maintain high availability with minimal downtime.
- The system should ensure automatic daily database backups.
- The system should ensure transactional integrity during payment processing.

Scalability

- The system should support future integration with SMS and WhatsApp notifications.
- The system should support scaling to accommodate increased numbers of users and bookings.

Usability

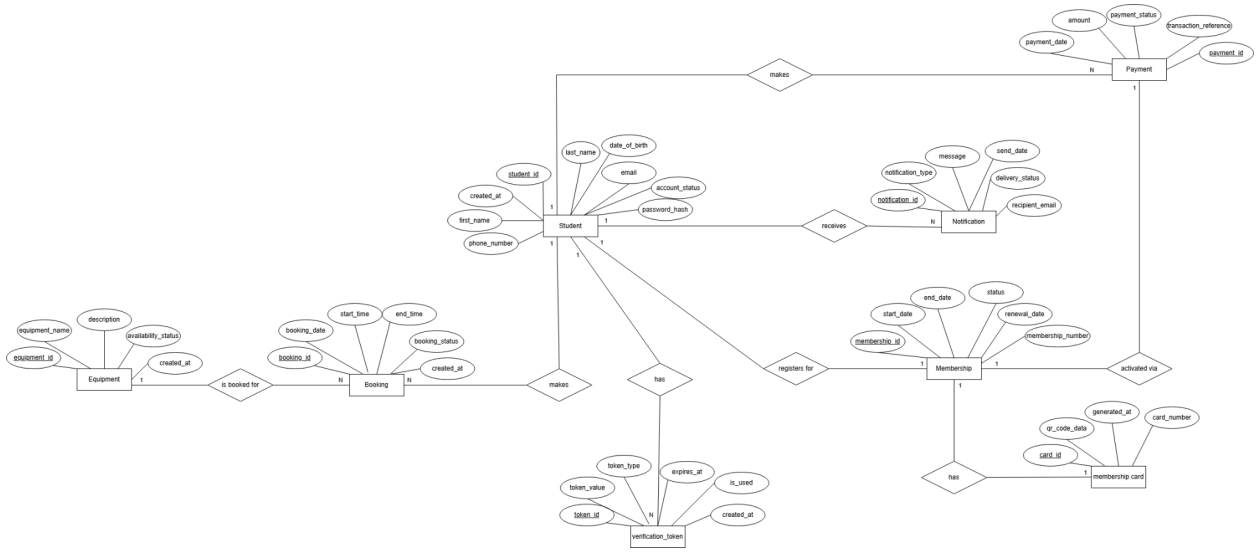
- The system should provide a responsive and mobile-friendly user interface.
- The system should display clear validation and error messages to users.

Maintainability

- The system should follow a modular and layered architecture design.
- The system should provide API documentation for all exposed endpoints.
- The system should allow easy updates and enhancements without affecting core functionality.

2. ER Diagram and Tables

ER Diagram



Tables

Student

Student (student_id: String **(PK)**, first_name: String, last_name: String, date_of_birth: Date, email: String, password_hash: String, phone_number: String, account_status: String, created_at: DateTime)

Payment

Payment (payment_id: Integer **(PK)**, student_id: String **(FK)**, amount: Decimal, payment_date: DateTime, payment_status: String, transaction_reference: String)

Membership

Membership (membership_id: Integer **(PK)**, student_id: String **(FK)**, payment_id: Integer **(FK)**, membership_number: String, start_date: Date, end_date: Date, renewal_date: Date, status: String)

Membership Card

Membership_Card (card_id: Integer **(PK)**, membership_id: Integer **(FK)**, card_number: String, qr_code_data: String, generated_at: DateTime)

Equipment

Equipment (equipment_id: Integer **(PK)**, equipment_name: String, description: String, availability_status: String, created_at: DateTime)

Booking

Booking (booking_id: Integer **(PK)**, student_id: String **(FK)**, equipment_id: Integer **(FK)**, booking_date: Date, start_time: String, end_time: String, booking_status: String, created_at: DateTime)

Notification

Notification (notification_id: Integer **(PK)**, student_id: String **(FK)**, notification_type: String, message: String, recipient_email: String, send_date: DateTime, delivery_status: String)

Verification Token

Verification_Token (token_id: Integer **(PK)**, student_id: String **(FK)**, token_value: String, token_type: String, expires_at: DateTime, is_used: Boolean, created_at: DateTime)

Phase 2.0

CSI345 Phase 1.0 Last Part

Identify all micro services required for the problem above

New Microservices

1. Student Registration Service
2. Membership Management Service
3. Equipment Booking Service
4. Payment Service
5. Notification Service
6. Membership Card Service
7. Reporting Service
8. Renewal Management Service

Existing Microservices (External Systems)

1. SARMS Student Validation API
2. Keycloak Identity and Access Management (IAM)
3. Payment Gateway Service (Stripe)
4. Email Service (SMTP)
5. SMS Notification Service (Future Integration)
6. WhatsApp Notification Service (Future Integration)

New Microservices

1. **Student Registration Service**

Endpoints:
POST /registrations
GET /registrations/{registrationId}
POST /registrations/{registrationId}/confirm-email
POST /registrations/resend-verification

OpenAPI specification:

```yaml
openapi: 3.0.3
info:
  title: Student Registration Service API
  version: 1.0.0

paths:
  /registrations:
    post:
      summary: Submit new student registration
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [studentNumber, firstName, lastName, dateOfBirth, universityEmail, password]
              properties:
                studentNumber:
                  type: string
                firstName:
                  type: string
                lastName:
                  type: string
                dateOfBirth:
                  type: string
                  format: date
                universityEmail:
                  type: string
                password:
                  type: string
      responses:
        '201':
          description: Registration successful and verification email sent
          content:
            application/json:
              schema:
                type: object
                properties:
                  registrationId:
                    type: string
                  message:
                    type: string
        '400':
          description: SARMS validation failed
          content:
            application/json:
              schema:
                type: object
                properties:
                  error:
                    type: string
                  message:
                    type: string
        '409':
          description: Student already registered
          content:
            application/json:
              schema:
                type: object
                properties:
                  error:
                    type: string
                  message:
                    type: string
```

```yaml
/registrations/{registrationId}:
  get:
    summary: Get registration status by ID
    parameters:
      - name: registrationId
        in: path
        required: true
        schema:
          type: string
    responses:
      '200':
        description: Registration details returned
        content:
          application/json:
            schema:
              type: object
              properties:
                registrationId:
                  type: string
                studentNumber:
                  type: string
                universityEmail:
                  type: string
                status:
                  type: string
                createdAt:
                  type: string
                  format: date-time
      '404':
        description: Registration not found
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
```

```yaml
/registrations/{registrationId}/confirm-email:
  post:
    summary: Confirm email using verification token
    parameters:
      - name: registrationId
        in: path
        required: true
        schema:
          type: string
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [token]
            properties:
              token:
                type: string
    responses:
      '200':
        description: Email verified successfully
        content:
          application/json:
            schema:
              type: object
              properties:
                message:
                  type: string
                nextStep:
                  type: string
      '400':
        description: Token expired
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
      '410':
        description: Token already used
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
      '404':
        description: Registration not found
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
```

```yaml
/registrations/resend-verification:
  post:
    summary: Resend verification email
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [universityEmail]
            properties:
              universityEmail:
                type: string
    responses:
      '200':
        description: Verification email resent successfully
        content:
          application/json:
            schema:
              type: object
              properties:
                message:
                  type: string
      '404':
        description: No pending registration found for this email
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
      '429':
        description: Too many resend requests
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: strings
```

2. Membership Management Services

Endpoints:

POST /memberships

GET /members
GET /members/{memberId}
GET /members/by-email/{email}

PATCH /members/{memberId}/block
PATCH /members/{memberId}/unblock

OpenAPI specification:

```yaml
openapi: 3.0.3
info:
  title: Membership Management Service API
  version: 1.0.0

paths:
  /memberships:
    post:
      summary: Create a new membership after successful payment
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [studentId, paymentId]
              properties:
                studentId:
                  type: string
                paymentId:
                  type: integer
      responses:
        '201':
          description: Membership created and activated
          content:
            application/json:
              schema:
                type: object
                properties:
                  membershipId:
                    type: integer
                  membershipNumber:
                    type: string
                  studentId:
                    type: string
                  startDate:
                    type: string
                    format: date
                  endDate:
                    type: string
                    format: date
                  status:
                    type: string
        '409':
          description: Active membership already exists
          content:
            application/json:
              schema:
                type: object
                properties:
                  error:
                    type: string
                  message:
                    type: string
        '404':
          description: Student or payment not found
          content:
            application/json:
              schema:
                type: object
                properties:
                  error:
                    type: string
                  message:
                    type: string
```

```yaml
/members:
  get:
    summary: List all members
    parameters:
      - name: status
        in: query
        required: false
        schema:
          type: string
          enum: [active, expired, blocked, pending]
    responses:
      '200':
        description: List of members returned
        content:
          application/json:
            schema:
              type: array
              items:
                type: object
                properties:
                  memberId:
                    type: string
                  firstName:
                    type: string
                  lastName:
                    type: string
                  email:
                    type: string
                  membershipNumber:
                    type: string
                  status:
                    type: string
                  endDate:
                    type: string
                    format: date
```

```yaml
/members/{memberId}:
  get:
    summary: Get member details by ID
    parameters:
      - name: memberId
        in: path
        required: true
        schema:
          type: string
    responses:
      '200':
        description: Member details returned
        content:
          application/json:
            schema:
              type: object
              properties:
                memberId:
                  type: string
                firstName:
                  type: string
                lastName:
                  type: string
                email:
                  type: string
                membershipNumber:
                  type: string
                startDate:
                  type: string
                  format: date
                endDate:
                  type: string
                  format: date
                status:
                  type: string
      '404':
        description: Member not found
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
```

```yaml
/members/by-email/{email}:
  get:
    summary: Get member by email address
    parameters:
      - name: email
        in: path
        required: true
        schema:
          type: string
    responses:
      '200':
        description: Member found and returned
        content:
          application/json:
            schema:
              type: object
              properties:
                memberId:
                  type: string
                firstName:
                  type: string
                lastName:
                  type: string
                email:
                  type: string
                membershipNumber:
                  type: string
                status:
                  type: string
      '404':
        description: No member found with this email
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
```

```yaml
/members/{memberId}/block:
  patch:
    summary: Block a member account
    parameters:
      - name: memberId
        in: path
        required: true
        schema:
          type: string
    responses:
      '200':
        description: Member blocked successfully
        content:
          application/json:
            schema:
              type: object
              properties:
                memberId:
                  type: string
                status:
                  type: string
                message:
                  type: string
      '404':
        description: Member not found
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
      '400':
        description: Member is already blocked
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
```

```yaml
/members/{memberId}/unblock:
  patch:
    summary: Unblock a member account
    parameters:
      - name: memberId
        in: path
        required: true
        schema:
          type: string
    responses:
      '200':
        description: Member unblocked successfully
        content:
          application/json:
            schema:
              type: object
              properties:
                memberId:
                  type: string
                status:
                  type: string
                message:
                  type: string
      '404':
        description: Member not found
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
      '400':
        description: Member is not blocked
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
```

3. Equipment Booking Service

Endpoints:

GET /equipment
GET /equipment/{equipmentId}/availability
POST /bookings
GET /bookings
GET /bookings/{bookingId}
PATCH /bookings/{bookingId}/cancel

OpenAPI specification:

```yaml
openapi: 3.0.3
info:
  title: Equipment Booking Service API
  version: 1.0.0

paths:
  /equipment:
    get:
      summary: List all available equipment
      responses:
        '200':
          description: Equipment list returned
          content:
            application/json:
              schema:
                type: array
                items:
                  type: object
                  properties:
                    equipmentId:
                      type: integer
                    equipmentName:
                      type: string
                    description:
                      type: string
                    availabilityStatus:
                      type: string
```

```yaml
/equipment/{equipmentId}/availability:
  get:
    summary: Get available time slots for an equipment item
    parameters:
      - name: equipmentId
        in: path
        required: true
        schema:
          type: integer
      - name: dateFrom
        in: query
        required: true
        schema:
          type: string
          format: date
      - name: dateTo
        in: query
        required: false
        schema:
          type: string
          format: date
    responses:
      '200':
        description: Availability slots returned
        content:
          application/json:
            schema:
              type: object
              properties:
                equipmentId:
                  type: integer
                equipmentName:
                  type: string
                slots:
                  type: array
                  items:
                    type: object
                    properties:
                      date:
                        type: string
                        format: date
                      startTime:
                        type: string
                      endTime:
                        type: string
                      status:
                        type: string
      '404':
        description: Equipment not found
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
```

```yaml
/bookings:
  post:
    summary: Create a new equipment booking
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [studentId, equipmentId, bookingDate, startTime, endTime]
            properties:
              studentId:
                type: string
              equipmentId:
                type: integer
              bookingDate:
                type: string
                format: date
              startTime:
                type: string
              endTime:
                type: string
              notes:
                type: string
    responses:
      '201':
        description: Booking confirmed
        content:
          application/json:
            schema:
              type: object
              properties:
                bookingId:
                  type: integer
                bookingReference:
                  type: string
                bookingDate:
                  type: string
                  format: date
                startTime:
                  type: string
                endTime:
                  type: string
                bookingStatus:
                  type: string
      '403':
        description: Membership not active
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
```

```yaml
    '409':
      description: Booking conflict with existing reservation
      content:
        application/json:
          schema:
            type: object
            properties:
              error:
                type: string
              message:
                type: string
    '422':
      description: Maximum bookings reached
      content:
        application/json:
          schema:
            type: object
            properties:
              error:
                type: string
              message:
                type: string

get:
  summary: List all bookings
  responses:
    '200':
      description: Booking list returned
      content:
        application/json:
          schema:
            type: array
            items:
              type: object
              properties:
                bookingId:
                  type: integer
                bookingReference:
                  type: string
                studentId:
                  type: string
                equipmentId:
                  type: integer
                bookingDate:
                  type: string
                  format: date
                startTime:
                  type: string
                endTime:
                  type: string
                bookingStatus:
                  type: string
```

```yaml
/bookings/{bookingId}:
  get:
    summary: Get booking details by ID
    parameters:
      - name: bookingId
        in: path
        required: true
        schema:
          type: integer
    responses:
      '200':
        description: Booking details returned
        content:
          application/json:
            schema:
              type: object
              properties:
                bookingId:
                  type: integer
                bookingReference:
                  type: string
                studentId:
                  type: string
                equipmentId:
                  type: integer
                bookingDate:
                  type: string
                  format: date
                startTime:
                  type: string
                endTime:
                  type: string
                bookingStatus:
                  type: string
                notes:
                  type: string
      '404':
        description: Booking not found
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
```

```yaml
/bookings/{bookingId}/cancel:
  patch:
    summary: Cancel an existing booking
    parameters:
      - name: bookingId
        in: path
        required: true
        schema:
          type: integer
    responses:
      '200':
        description: Booking cancelled successfully
        content:
          application/json:
            schema:
              type: object
              properties:
                bookingId:
                  type: integer
                bookingStatus:
                  type: string
                message:
                  type: string
      '400':
        description: Cancellation window has passed
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
      '404':
        description: Booking not found
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
```

4. Payment Service

Endpoints:
POST /payments
GET /payments/{paymentId}
GET /payments/member/{memberId}
POST /payments/webhooks/gateway

OpenAPI specification

```yaml
openapi: 3.0.3
info:
  title: Payment Service API
  version: 1.0.0

paths:
  /payments:
    post:
      summary: Initiate a membership fee payment
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [studentId, paymentType]
              properties:
                studentId:
                  type: string
                paymentType:
                  type: string
                  enum: [new_membership, renewal]
                amount:
                  type: number
                currency:
                  type: string
      responses:
        '201':
          description: Payment session created
          content:
            application/json:
              schema:
                type: object
                properties:
                  paymentId:
                    type: integer
                  paymentReference:
                    type: string
                  amount:
                    type: number
                  currency:
                    type: string
                  paymentUrl:
                    type: string
                  status:
                    type: string
        '400':
          description: Account not verified
          content:
            application/json:
              schema:
                type: object
                properties:
                  error:
                    type: string
                  message:
                    type: string
        '409':
          description: Payment already in progress
          content:
            application/json:
              schema:
                type: object
                properties:
                  error:
                    type: string
                  message:
                    type: string
```

```yaml
/payments/{paymentId}:
  get:
    summary: Get payment details by ID
    parameters:
      - name: paymentId
        in: path
        required: true
        schema:
          type: integer
    responses:
      '200':
        description: Payment details returned
        content:
          application/json:
            schema:
              type: object
              properties:
                paymentId:
                  type: integer
                paymentReference:
                  type: string
                studentId:
                  type: string
                amount:
                  type: number
                currency:
                  type: string
                status:
                  type: string
                paymentType:
                  type: string
                paymentDate:
                  type: string
                  format: date-time
      '404':
        description: Payment not found
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
```

```yaml
/payments/member/{memberId}:
  get:
    summary: Get all payments for a member
    parameters:
      - name: memberId
        in: path
        required: true
        schema:
          type: string
    responses:
      '200':
        description: Member payment history returned
        content:
          application/json:
            schema:
              type: array
              items:
                type: object
                properties:
                  paymentId:
                    type: integer
                  paymentReference:
                    type: string
                  amount:
                    type: number
                  currency:
                    type: string
                  status:
                    type: string
                  paymentType:
                    type: string
                  paymentDate:
                    type: string
                    format: date-time
      '404':
        description: No payments found for this member
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
```

```yaml
/payments/webhooks/gateway:
  post:
    summary: Receive payment status notification from gateway
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [event, transactionReference, amount, timestamp]
            properties:
              event:
                type: string
                enum: [payment.success, payment.failed, refund.created]
              transactionReference:
                type: string
              amount:
                type: number
              currency:
                type: string
              timestamp:
                type: string
                format: date-time
    responses:
      '200':
        description: Webhook received and acknowledged
        content:
          application/json:
            schema:
              type: object
              properties:
                received:
                  type: boolean
      '400':
        description: Invalid webhook signature
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
```

5. Notification Service

Endpoints:
POST /notifications/send
POST /notifications/{notificationId}
POST /notifications/student/{studentId}

POST /notifications/{notificationId}/retry

OpenAPI specification:

```yaml
openapi: 3.0.3
info:
  title: Notification Service API
  version: 1.0.0

paths:
  /notifications/send:
    post:
      summary: Queue a notification for delivery
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [studentId, notificationType, recipientEmail]
              properties:
                studentId:
                  type: string
                notificationType:
                  type: string
                  enum:
                    - verification
                    - password_reset
                    - 2fa_otp
                    - payment_confirmation
                    - booking_confirmation
                    - booking_reminder
                    - renewal_reminder
                    - account_blocked
                    - renewal_link
                recipientEmail:
                  type: string
                templateData:
                  type: object
                scheduledAt:
                  type: string
                  format: date-time
      responses:
        '202':
          description: Notification queued for delivery
          content:
            application/json:
              schema:
                type: object
                properties:
                  notificationId:
                    type: integer
                  status:
                    type: string
                  scheduledAt:
                    type: string
                    format: date-time
        '400':
          description: Invalid notification type or missing required fields
          content:
            application/json:
              schema:
                type: object
                properties:
                  error:
                    type: string
                  message:
                    type: string
```

```yaml
/notifications/{notificationId}:
  get:
    summary: Get notification record and delivery status
    parameters:
      - name: notificationId
        in: path
        required: true
        schema:
          type: integer
    responses:
      '200':
        description: Notification details returned
        content:
          application/json:
            schema:
              type: object
              properties:
                notificationId:
                  type: integer
                notificationType:
                  type: string
                recipientEmail:
                  type: string
                status:
                  type: string
                sendDate:
                  type: string
                  format: date-time
                retryCount:
                  type: integer
      '404':
        description: Notification not found
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
```

```yaml
/notifications/student/{studentId}:
  get:
    summary: List all notifications for a student
    parameters:
      - name: studentId
        in: path
        required: true
        schema:
          type: string
    responses:
      '200':
        description: Notification history returned
        content:
          application/json:
            schema:
              type: array
              items:
                type: object
                properties:
                  notificationId:
                    type: integer
                  notificationType:
                    type: string
                  status:
                    type: string
                  sendDate:
                    type: string
                    format: date-time
      '404':
        description: No notifications found for this student
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
```

```yaml
/notifications/{notificationId}/retry:
  post:
    summary: Manually retry a failed notification
    parameters:
      - name: notificationId
        in: path
        required: true
        schema:
          type: integer
    responses:
      '200':
        description: Notification queued for retry
        content:
          application/json:
            schema:
              type: object
              properties:
                notificationId:
                  type: integer
                status:
                  type: string
                message:
                  type: string
      '400':
        description: Maximum retries reached
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
      '404':
        description: Notification not found
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
```

6. **Membership card Service**

   POST/cards

   GET /cards/{cardId}

   GET /cards/member/{memberId}

   GET /cards/{cardId}/download

   GET /cards/verify/{qrPayload}

OpenAPI Specification:

```yaml
openapi: 3.0.3
info:
  title: Membership Card Service API
  version: 1.0.0

paths:
  /cards:
    post:
      summary: Generate a membership card for an active membership
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [memberId, firstName, lastName, membershipId]
              properties:
                memberId:
                  type: string
                firstName:
                  type: string
                lastName:
                  type: string
                membershipId:
                  type: integer
      responses:
        '201':
          description: Membership card generated successfully
          content:
            application/json:
              schema:
                type: object
                properties:
                  cardId:
                    type: string
                  memberId:
                    type: string
                  membershipNumber:
                    type: string
                  issueDate:
                    type: string
                    format: date
                  expiryDate:
                    type: string
                    format: date
        '404':
          description: Membership not found
          content:
            application/json:
              schema:
                type: object
                properties:
                  error:
                    type: string
                  message:
                    type: string
        '409':
          description: Card already exists for this membership
          content:
            application/json:
              schema:
                type: object
                properties:
                  error:
                    type: string
                  message:
                    type: string
```

```yaml
/cards/{cardId}:
  get:
    summary: Get membership card details by ID
    parameters:
      - name: cardId
        in: path
        required: true
        schema:
          type: string
    responses:
      '200':
        description: Membership card details returned
        content:
          application/json:
            schema:
              type: object
              properties:
                cardId:
                  type: string
                memberId:
                  type: string
                firstName:
                  type: string
                lastName:
                  type: string
                membershipNumber:
                  type: string
                issueDate:
                  type: string
                  format: date
                expiryDate:
                  type: string
                  format: date
                status:
                  type: string
      '404':
        description: Card not found
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
```

```yaml
/cards/member/{memberId}:
  get:
    summary: Get card by member ID
    parameters:
      - name: memberId
        in: path
        required: true
        schema:
          type: string
    responses:
      '200':
        description: Membership card returned
        content:
          application/json:
            schema:
              type: object
              properties:
                cardId:
                  type: string
                memberId:
                  type: string
                membershipNumber:
                  type: string
                issueDate:
                  type: string
                  format: date
                expiryDate:
                  type: string
                  format: date
                status:
                  type: string
      '404':
        description: No card found for this member
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
```

```yaml
/cards/{cardId}/download:
  get:
    summary: Download membership card as PDF
    parameters:
      - name: cardId
        in: path
        required: true
        schema:
          type: string
    responses:
      '200':
        description: PDF file returned for download
        content:
          application/pdf:
            schema:
              type: string
              format: binary
      '403':
        description: Access denied
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
      '404':
        description: Card not found
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
```

```yaml
/cards/verify/{qrPayload}:
  get:
    summary: Verify a membership card using its QR code payload
    parameters:
      - name: qrPayload
        in: path
        required: true
        schema:
          type: string
    responses:
      '200':
        description: Verification result returned
        content:
          application/json:
            schema:
              type: object
              properties:
                valid:
                  type: boolean
                memberId:
                  type: string
                fullName:
                  type: string
                membershipNumber:
                  type: string
                status:
                  type: string
                expiryDate:
                  type: string
                  format: date
      '400':
        description: Invalid or tampered QR payload
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
      '404':
        description: Member not found
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
```

7. **Reporting service**

GET /reports/memberships

GET /reports/bookings

GET /reports/payments

GET /reports/financial-summary

GET /reports/audit

OpenAPI Specification:

```yaml
openapi: 3.0.3
info:
  title: Reporting Service API
  version: 1.0.0

paths:
  /reports/memberships:
    get:
      summary: Generate membership report
      parameters:
        - name: dateFrom
          in: query
          required: false
          schema:
            type: string
            format: date
        - name: dateTo
          in: query
          required: false
          schema:
            type: string
            format: date
        - name: format
          in: query
          required: false
          schema:
            type: string
            enum: [json, pdf, csv]
            default: json
      responses:
        '200':
          description: Membership statistics returned
          content:
            application/json:
              schema:
                type: object
                properties:
                  totalMembers:
                    type: integer
                  activeMembers:
                    type: integer
                  expiredMembers:
                    type: integer
                  blockedMembers:
                    type: integer
                  newRegistrations:
                    type: integer
        '403':
          description: Admin access required
          content:
            application/json:
              schema:
                type: object
                properties:
                  error:
                    type: string
                  message:
                    type: string
```

```yaml
/reports/bookings:
  get:
    summary: Generate booking report
    parameters:
      - name: dateFrom
        in: query
        required: false
        schema:
          type: string
          format: date
      - name: dateTo
        in: query
        required: false
        schema:
          type: string
          format: date
      - name: format
        in: query
        required: false
        schema:
          type: string
          enum: [json, pdf, csv]
          default: json
    responses:
      '200':
        description: Booking statistics returned
        content:
          application/json:
            schema:
              type: object
              properties:
                totalBookings:
                  type: integer
                activeBookings:
                  type: integer
                cancelledBookings:
                  type: integer
                completedBookings:
                  type: integer
      '403':
        description: Admin access required
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
```

```yaml
/reports/payments:
  get:
    summary: Generate payment report
    parameters:
      - name: dateFrom
        in: query
        required: false
        schema:
          type: string
          format: date
      - name: dateTo
        in: query
        required: false
        schema:
          type: string
          format: date
      - name: format
        in: query
        required: false
        schema:
          type: string
          enum: [json, pdf, csv]
          default: json
    responses:
      '200':
        description: Payment statistics returned
        content:
          application/json:
            schema:
              type: object
              properties:
                totalPayments:
                  type: integer
                successfulPayments:
                  type: integer
                failedPayments:
                  type: integer
                totalRevenue:
                  type: number
      '403':
        description: Admin access required
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
```

```yaml
/reports/financial-summary:
  get:
    summary: Generate financial summary report
    parameters:
      - name: dateFrom
        in: query
        required: false
        schema:
          type: string
          format: date
      - name: dateTo
        in: query
        required: false
        schema:
          type: string
          format: date
      - name: format
        in: query
        required: false
        schema:
          type: string
          enum: [json, pdf, csv]
          default: json
    responses:
      '200':
        description: Financial overview returned
        content:
          application/json:
            schema:
              type: object
              properties:
                totalRevenue:
                  type: number
                totalMembers:
                  type: integer
                totalBookings:
                  type: integer
                revenueByMonth:
                  type: array
                  items:
                    type: object
                    properties:
                      month:
                        type: string
                      revenue:
                        type: number
      '403':
        description: Admin access required
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
```

```yaml
/reports/audit:
  get:
    summary: Generate system audit log report
    parameters:
      - name: dateFrom
        in: query
        required: false
        schema:
          type: string
          format: date
      - name: dateTo
        in: query
        required: false
        schema:
          type: string
          format: date
      - name: format
        in: query
        required: false
        schema:
          type: string
          enum: [json, pdf, csv]
          default: json
    responses:
      '200':
        description: Audit log returned
        content:
          application/json:
            schema:
              type: array
              items:
                type: object
                properties:
                  eventId:
                    type: integer
                  action:
                    type: string
                  performedBy:
                    type: string
                  affectedStudentId:
                    type: string
                  timestamp:
                    type: string
                    format: date-time
      '403':
        description: Admin access required
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
```

8. Renewal Management Service

Endpoints:
POST /renewals
GET /renewals/{renewalId}

POST /renewals/validate-token
POST /renewals/{renewalId}/complete

OpenAPI specification:

```yaml
openapi: 3.0.3
info:
  title: Renewal Management Service API
  version: 1.0.0

paths:
  /renewals:
    post:
      summary: Initiate renewal - blocked member submits email to receive renewal link
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [email]
              properties:
                email:
                  type: string
      responses:
        '201':
          description: Renewal request created and link sent to email
          content:
            application/json:
              schema:
                type: object
                properties:
                  renewalId:
                    type: string
                  message:
                    type: string
        '400':
          description: Email format is invalid
          content:
            application/json:
              schema:
                type: object
                properties:
                  error:
                    type: string
                  message:
                    type: string
        '404':
          description: No blocked account found for this email
          content:
            application/json:
              schema:
                type: object
                properties:
                  error:
                    type: string
                  message:
                    type: string
```

```yaml
/renewals/{renewalId}:
  get:
    summary: Get renewal request details by ID
    parameters:
      - name: renewalId
        in: path
        required: true
        schema:
          type: string
    responses:
      '200':
        description: Renewal details returned
        content:
          application/json:
            schema:
              type: object
              properties:
                renewalId:
                  type: string
                email:
                  type: string
                studentId:
                  type: string
                status:
                  type: string
                createdAt:
                  type: string
                  format: date-time
                expiresAt:
                  type: string
                  format: date-time
      '404':
        description: Renewal request not found
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
```

```yaml
/renewals/validate-token:
  post:
    summary: Validate renewal token before showing payment screen
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [token]
            properties:
              token:
                type: string
    responses:
      '200':
        description: Token is valid - student can proceed to payment
        content:
          application/json:
            schema:
              type: object
              properties:
                valid:
                  type: boolean
                studentId:
                  type: string
                email:
                  type: string
                expiresAt:
                  type: string
                  format: date-time
      '400':
        description: Token has expired
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
      '410':
        description: Token has already been used
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
```

```yaml
/renewals/{renewalId}/complete:
  post:
    summary: Complete renewal after successful payment
    parameters:
      - name: renewalId
        in: path
        required: true
        schema:
          type: string
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [studentId, paymentId]
            properties:
              studentId:
                type: string
              paymentId:
                type: integer
    responses:
      '200':
        description: Membership renewed successfully
        content:
          application/json:
            schema:
              type: object
              properties:
                membershipId:
                  type: integer
                membershipNumber:
                  type: string
                newStartDate:
                  type: string
                  format: date
                newEndDate:
                  type: string
                  format: date
                status:
                  type: string
      '400':
        description: Payment not confirmed
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
      '404':
        description: Renewal request not found
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
      '409':
        description: Membership already active
        content:
          application/json:
            schema:
              type: object
              properties:
                error:
                  type: string
                message:
                  type: string
```

Existing Micro Services

1. SARMS Student Validation API

Endpoint: GET /api/students/{student_id}
Description
Retrieves the official student record from the SARMS system using the student ID. The system uses this endpoint during registration to verify that the student exists and that their details match university records.

Endpoint: POST /api/students/validate
Description
Accepts the submitted details and returns a match result.

2. Keycloak Identity and Access Management (IAM)

Endpoint: POST /realms/{realm}/protocol/openid-connect/token
Description
Authenticates a user and generates an access token used to access protected system APIs.

Endpoint: GET /realms/{realm}/protocol/openid-connect/userinfo
Description
Returns information about the currently authenticated user.

Endpoint: POST /admin/realms/{realm}/users
Description
Creates a new user account in Keycloak during student registration.

Endpoint: PUT /admin/realms/{realm}/users/{user_id}/reset-password
Description
Allows resetting the password for an existing user.

Endpoint: POST /realms/{realm}/protocol/openid-connect/token/introspect
Description
Validates and checks the status of an access token to ensure secure access to protected APIs.

3. Payment Gateway Service (Stripe)

Endpoint: POST /v1/payment_intents
Description
Creates a payment transaction for the membership fee.

Endpoint: GET /v1/payment_intents/{payment_id}
Description
Retrieves the status of a payment to confirm whether the transaction was successful.

4. Email Service (SMTP)

Endpoint: POST /send-email
Description
Sends system emails such as account verification, password reset links, OTP codes, payment confirmations, and booking reminders.

5. SMS Notification Service (Future Integration)

Endpoint: POST /sms/send
Description
Sends SMS notifications to users for alerts such as OTP verification or booking reminders.

6. WhatsApp Notification Service (Future Integration)

Endpoint: POST /whatsapp/send
Description
Sends notifications to users through WhatsApp for booking confirmations and membership reminders.

Katlego and Kago Tasks:

- ER diagram + tables in BCNF.
- List of new micro services, their endpoints, and OpenAPI specification for each end point.

Tagala Mannathoko Tasks:

- Implement the database design in Oracle.
- List of existing micro services and the end points required in this solution. Describe each end point.

Marang Ponatshego Tasks:

- Detailed requirements.
- Identification of all micro services required for the problem.

Marang Ponatshego 202204203
Kevin Mannathoko 202203462
Katlego Masike 202207210
Kago Phuthego 202207201